PSA Group AEE/CAN2004 Can-bus Clusters Prototyping for Simracing purposes

Using a Peugeot 407 Cluster as a simracing accessory

RYM88 – Ifrim Matei

Contents

Youtube video proof of concept :

https://youtu.be/AXVqf_LW00Q?is=hvC2KXirDKdAgy4Z

Introduction & Motivation

After the previous retrofits I have done on my car, I was left with the 407's original cluster laying around, so I thought it would be to put it to better use than laying around gathering dust.

The passion for sim racing was passed down from me to my younger brother, so he basically inherited a full, equipped with off the shelf hardware, racing setup, complete with Thrustmaster wall mounted pedals, 8 speed manual shifter, a T300RS steering wheel base, generic Aliexpress handbrake and to hold everything together a Playseat rig.

So naturally, seeing all the previous experience I have with PSA, Stellantis and VAG group hardware components and CANBUS systems (CANBUS is important to mention as it is the sole communication protocol used in the PSA AEE2004/CAN2004 clusters), I wanted to provide him with something to further enhance the experience, probably the first of many things to come, so I started working on a prototype to make the cluster work with the Simhub platform.

I. Materials and tools

Keeping in mind this is just a prototype, I decided a more fast, simple to implement and rewards based approach.

I.I. Materials list

- Peugeot 407 Diesel Cluster Gauge, manufactured by VDO, p/n 9658138280 (mention: keep in mind that the actual p/n does not matter, any cluster from the AEE2004/CAN2004 generation will be suitable for this purpose, afaik as long as it is part of the crossrange/evolution range of BSI's)
- Groundstudio Jade U1, a devboard designed and manufactured locally, in Sibiu, Romania, powered by the Atmega328p, with USB type C connectivity, a big plus which I consider to be an standard in these times for connectivity and interusability. I am a big fan of these boards as they are reliable, use good quality components, and of course, they are local-based, so it is also an investment in further development of the local industry and the sustainability behind it. Most likely it is going to be replaced with a uno type board in the final version, altough I might also include a version that is solely based on shields like this one is for ease of repeatability for those who do not posses the tools and knowledge needed to build a smaller package for this purpose.
- Sparkfun CANBUS Shield, red revision, based on the popular MCP2515 chip; this is a board I had laying around from a previous project initiative that I never got around to finish, so it has been used for this prototype.
- Generic 12V 5A power supply with a barrel jack connector, of course used for the convenience it brings for a prototype, but might end up being the go-to solution for power delivery in the final version, as it is easy and cheap to acquire and provides enough power for the cluster, devboard, shield and any other devices that could be connected in the future.
- The original TE Connectivity 18 pin cluster connector that is found in the OE cable.
- 2 Dupont (for the MCP shield) and 4 TE Connectivity p/n 5-962885-1 (for the cluster connector) terminated wires.
- Female barell jack screw type connector for 12V input, great for ease of use (no soldering needed, as that would render some materials to be able to be used just one time, and I strongly abide to the fact that there should be as less waste as possible)
- Male barell jack power connector in order to power the devboard; no stepdown needed as the devboard supports up to 13v input via the barrel jack
- 2 Wago type connectors, through which the CAN L and CAN H pass, a nice way for the prototype to easily accommodate any other canbus modules that might be added afterwards.

I.II. Tools

- Standard Phillips screwdriver bit for tightening the male screw type terminal barrel jack

- Small bit for removing the old pins from the cluster connector, so as to achieve less wire clutter
- Hot glue gun – great for a temporary mount of the components to the cluster, so as to achieve a less cluttered and usable hardware layout.
- Electrical tape – used to protect against shorts between components in prone areas.
- Label printer – great for keeping track of the wires, so as to maintain a clean and precise work environment.

II. Protocols and hardware specifications

The Peugeot 407, AEE/CAN2010 cluster is primarily running off a canbus communication protocol, mainly the CAN Confort (as labeled by the manufacturer) 125kbps network.

In order to establish communication between the devboard and computer, I used the standard serial port available on the board via the usb c cable connection, between the devboard and the cluster, we need to use a MCP2515 shield board, that enables the devboard to send digital signals to the canbus components, those further relaying the CAN id’s to the clusters onboard TJA chip.

III. Software

Looking for the most efficiency possible, already having a knowledge of the CANBUS system we are dealing with and adjacent repositories, I used the AI service Claude[1] for code generation.

Very important and timesaving data was extracted from Github users Ludwig V a.k.a. Vlud[2] (a pioneer in AEE2004,2007,2010 and 2020 protocols, known for his contributions in creating powerful telecoding, module accesing and can gateway tools, including the popular CAN2004 to CAN2010 repo used for many retrofits on PSA era cars) and PROTOTUX[3] (another great contributor in PSA CANBUS systems, having a whole repository of AEE2004 reverse engineered protocols)

The prompt, which is generated in Romanian, can be viewed at https://claude.ai/share/f31b50cd-0e19-4bdc-abdd-73b095a8f7f9.

Further work will be done in order to acquire most if not all functionality off the cluster.

Full Code that needs to be uploaded to the devboard:

cluster_407_simhub .ino

Full code that needs to be introduced in the Simhub app:

Simhub setup.txt

[1] https://claude.ai

[2] https://github.com/ludwig-v

[3] https://github.com/prototux

IV. Setup & Assembly

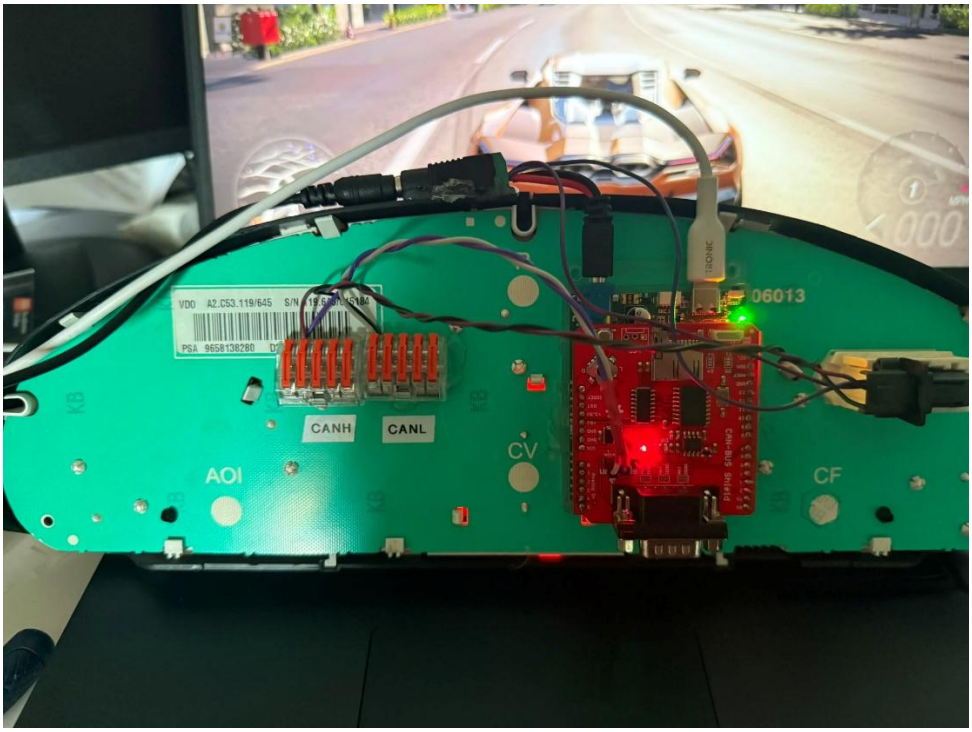
IV.I. Hardware assembly

We are going to start by attaching the shield to the devboard, then sticking the underside of the devboard transparent holder on the back PCB of the cluster using hot glue. In this step we also glue down the female barrel jack connector, and the two Wago style connectors to the top, respectively the back of the cluster assembly, and label the Wago connectors with CANL and CANH using the label printer, so as to keep everything as tidy as possible.

Then comes the 12V power feed: using the barrel jack connectors, the female one of which I glued to the top of the cluster, we took the positive (+) and negative (-) 12 V terminals of the male barrel jack connector, and the wired the Clusters power inputs according to PSA's Peugeot Sedre wiring diagram program which I accessed, with each of them having the cluster side terminated with the TE Connectivity pins, such as positive (+) 12V (Cluster PIN 16) and negative(-) (Cluster PIN 18).

The Canbus wiring was done by bringing 2 TE Connectivity terminated wires (each wire being terminated on the cluster side) to the cluster, more specifically PIN 1 of cluster connector - CAN High, and PIN 2 of the cluster connector – CAN Low, the other ends of the cables going to their according labeled Wago style connectors. Then, from the Wago style connectors, 2 wires were used, one for CAN High and one for CAN Low, each of them terminated with dupont wires on one side, so as to be connected to the CAN H and CAN L pins of the Sparkfun Canbus Shield.

Then, I proceeded to reinstall the TE-Connectivity clusters connector casing, plugged it in the cluster and proceeded to connect the 12V power from the power brick in order to test that the cluster, devboard and shield all light up and are able to be accessed. Finished product:

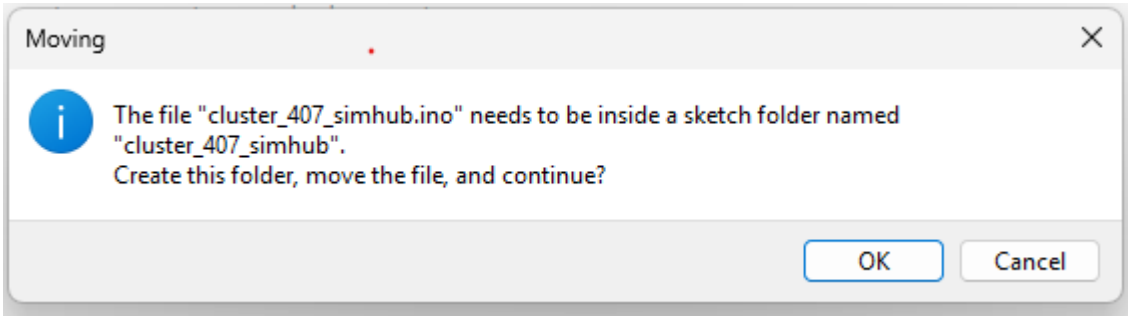


IV.II. Software setup

Arduino IDE and Devboard programming

Now, we are going to connect the devboard to the computer via the usb c cable, and using the Arduino IDE program we are going to write the code onto the devboard.

Start by opening the provided .ino file in the software section of the document, and proceed as follows:

- 

  Here you are going to select OK
- Install the "mcp_can" by coryjfowler , that can be found on https://github.com/coryjfowler/MCP_CAN_lib -> click on the green code button, and from the options select download ZIP -> on the arduino IDE program go to the top section, select Sketch, press Include Library and select Add .zip library, where a files page will pop up and you will need to select the downloaded .zip library from the place where you downloaded it, then press open and proceed installing it, should not take more than a few moments.
- Afterwards, you are ready to select your Uno style board on the top board selection menu, then press the upload button (->) and wait for the software to be uploaded to the arduino.
- After all those steps, you have oficially completed the arduino side code uploading

Simhub-side setup

- DISCLAIMER: Be sure to close the arduino IDE before starting these steps, as it will interfere with the process of setting things up.
- Start by installing Simhub from their official website[4]
- When the installation process is done, proceed by opening the simhub app
- On the left side of the user interface, press Add/remove features -> Search in the top bar Custom Serial Devices -> Turn it on using the slider on the right side.
- Now press the Additional plugins on the left side of the user interface, and select Add new serial device on the top right side. Now a device has been added, press the button next to the slider to fold down the settings. Select the serial port according to your devboard -> choose baudrate 115200 -> slide the Automatic reconnect option to on -> go to the Update messages -> press EDIT -> select use Javascript -> in the Javascript

[4] https://www.simhubdash.com

(the field under the Run once javascript code) -> copy-paste into that field the provided code in the software section of the document from the .txt file.

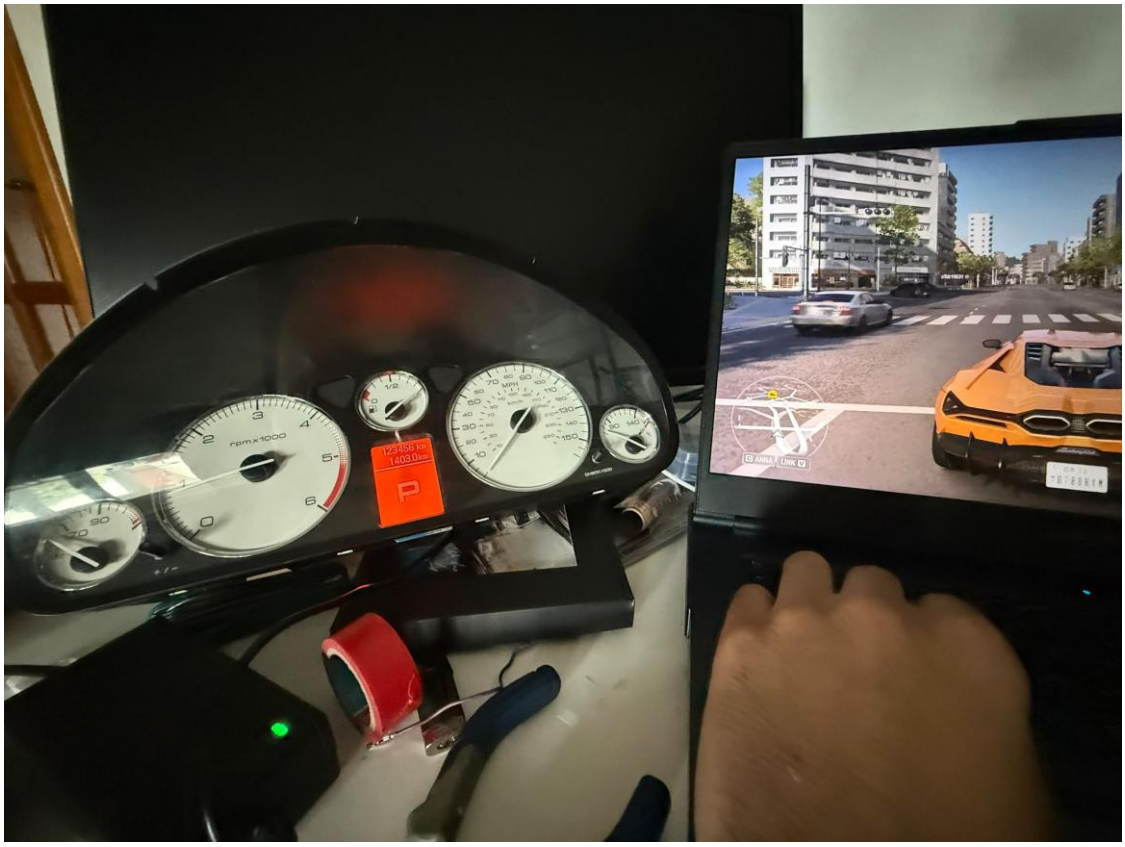
- Press Ok and you are officially done! All that is left is to turn on a game and enjoy!

V. Current functionality

As of the moment of writing this first prototype document, the following functions are present, accurate and fully working:

- Speedometer
- RPM Gauge
- Fuel level
- Left and right turn signals
- Coolant temperature
- Oil temperature

I will continue to work on improving both the hardware and software side of this project as to include more functionality in the future, such as the warning lights, central lcd notifications, and most likely I will use a COM2004 or similar unit that I have laying around in order to further increase the accuracy of the simracing rig, having fully functional stalks to control the cars features.



Conclusions

This is a proof of concept, prototype unit, so the functionality of the cluster will be further enhanced, the vision I aim for is building a whole simracing ready setup made of CAN/AEE2004 parts, that provides a cheap, easy to implement base for the simracing community.

I am very pleased with the results that were achieved in less than an hours work, and I will continue to enhance it and will keep in mind both more off the shelf solutions for those who are not technically inclined, that are easy to build and use, and also more compact and performant packages for those that posses or want to acquire technical knowledge.

Any feedback and contributions to this project will be warmly appreciated.

In the future I aim to modify the sim rig so to have a fully functional 407 dashboard in place, with working electronics such as a RNEG unit, lit up climate control panel, working 12V accesory port, maybe even using the 407's original pedal layout, meaning clutch, brake and accelerator, with force feedback on the clutch and brake, and hope to use an original steering column as to even fit true automotive wheels to it without straining the actual wheel base.